KP25 Boosting and AdaBoost: Turning Weak into Strong

> Level: undergraduate, year 3
>
> Pattern Recognition | Chapter 4, §4.22

Slides: read and download

Learning objectives

- Derive the AdaBoost weights and $\alpha_m = \frac{1}{2}\ln\frac{1-P_m}{P_m}$ and work one weight update by hand.
- Explain the margin explanation of AdaBoost's relative immunity to overfitting and the limits of that explanation.
- Explain why AdaBoost degrades on hard tasks with noisy labels.

KP25 Boosting and AdaBoost: Turning Weak into Strong

One-sentence summary: A classifier only a little better than guessing, retrained again and again on the samples it got wrong and combined by a weighted vote, can push the training error as low as you like; but boosting is no panacea.

1. Weak to strong

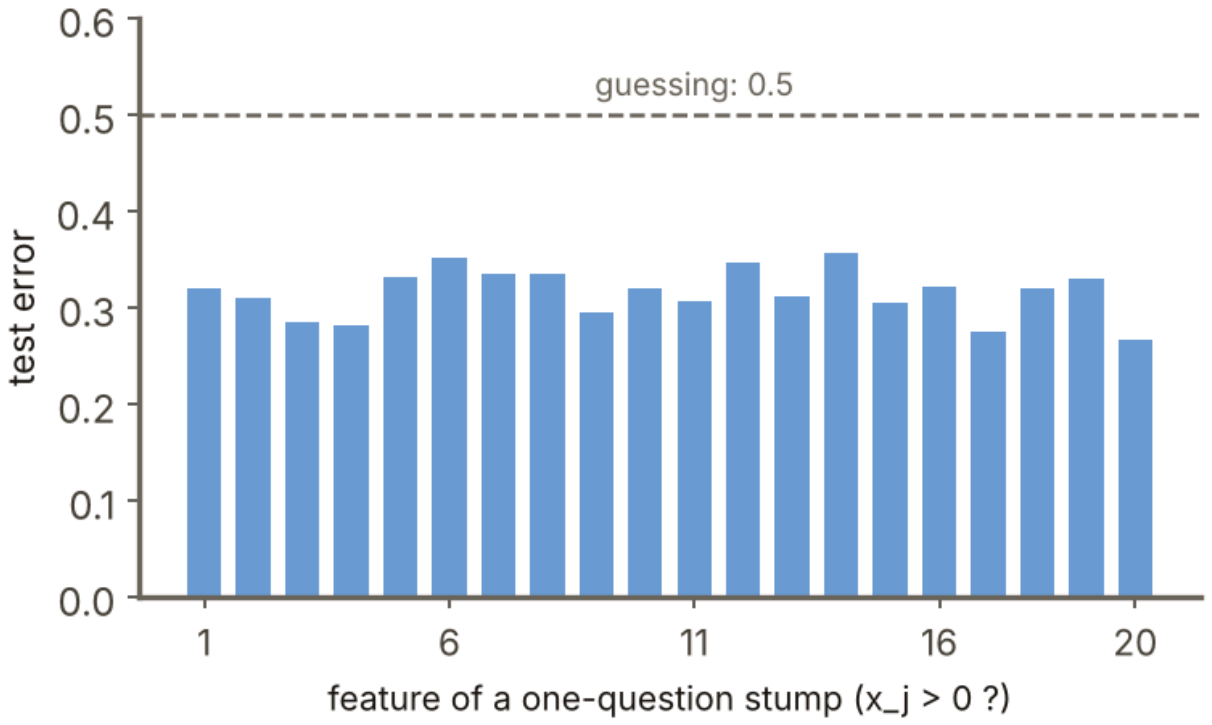
Plain explanation: Can a weak classifier be boosted into a strong one? (Labels $y \in \{\pm 1\}$ and classifier outputs $\phi \in \{\pm 1\}$.)

- **Weak classifier**: one whose error is only a little better than guessing (0.5).
- **Stump**: a tree that asks one question about one feature, such as "$x_j > 0$?".

Example (designed here): two 20-D Gaussians (means $\mp a$, $a = 2/\sqrt{20}$), 200 training, 400 test points; the 20 stumps "$x_j > 0$?" err 27% to 36%, mean 31.6%. The

mechanism: reweight the samples every round, giving hard (misclassified) points more weight, then take a weighted vote.

Figure `lec05-kp25-weak`: the test errors of the 20 stumps.



2. Where weights come from

Stage-wise optimization: optimize only the newly added classifier and freeze the earlier ones: $F_m = F_{m-1} + \alpha_m \phi_m$. Split stage by stage, the exponential loss (KP20) $\sum_i \exp(-y_i F(x_i))$ becomes

$$J = \sum_i w_i \exp\big(-y_i \alpha \phi(x_i)\big), \qquad w_i = \exp\big(-y_i F_{m-1}(x_i)\big).$$

w_i contains neither α nor ϕ, so it is the sample weight; ϕ_m then minimizes the weighted error P_m (weights normalized to sum 1).

Figure `lec05-kp25-wts`: the weights emerge from the exponential loss.

$$\text{minimize } \Sigma_i \exp(-y_i F_m(x_i)),\ F_m = F_{m-1} + \alpha \varphi$$

$$= \Sigma_i w_i \exp(-y_i \alpha \varphi(x_i))$$

$$w_i = \exp(-y_i F_{m-1}(x_i))$$

contains neither α nor φ: it is the sample weight

so φ must minimize the weighted error

$$P_m = \Sigma_i w_i \cdot [y_i \varphi(x_i) < 0]$$

weights are not a heuristic:

they fall out of the exponential loss

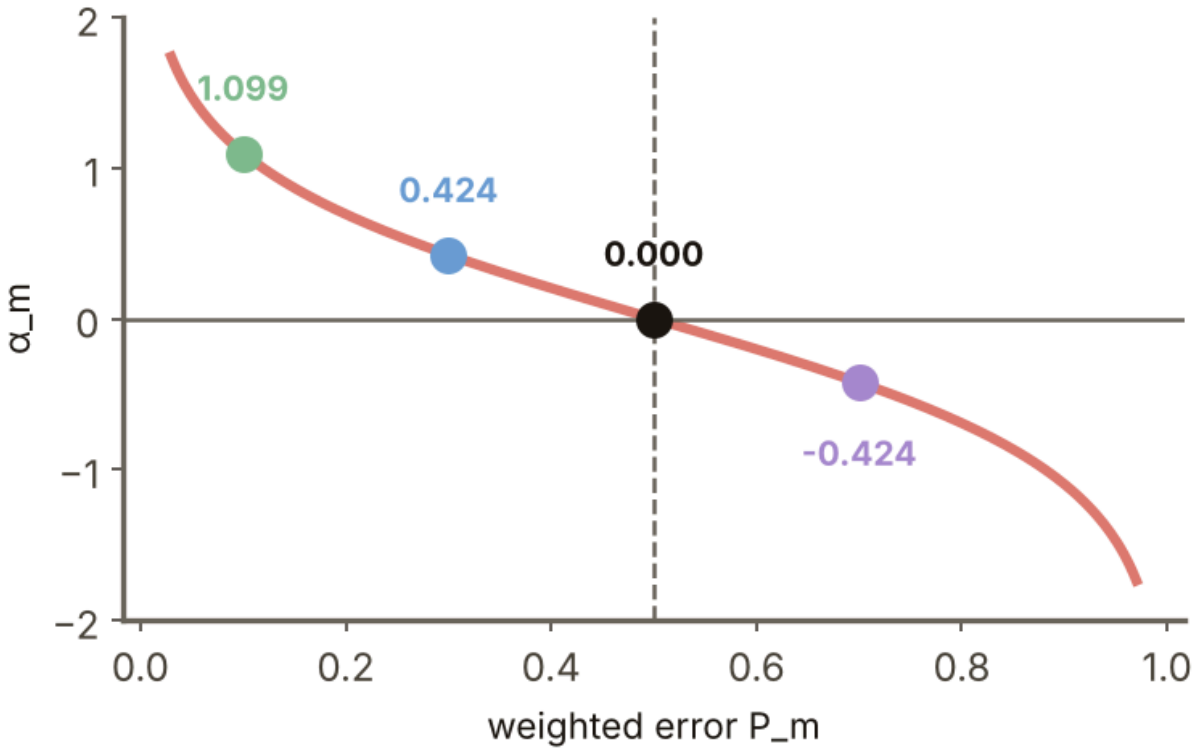
3. The voice α

Formal definition: back in J, $\alpha_m = \arg\min\{e^{-\alpha}(1 - P_m) + e^{\alpha} P_m\}$ and a zero derivative gives

$$\alpha_m = \tfrac{1}{2} \ln \frac{1 - P_m}{P_m}.$$

$P = 0.1$: 1.099; 0.3: 0.424; 0.5: 0; 0.7: -0.424. Accurate → a large voice; guessing → 0; worse than guessing ($P > 0.5$) → negative, use its answer inverted.

Figure `lec05-kp25-alpha`: α against the weighted error.



α = ½ ln((1 − P)/P)

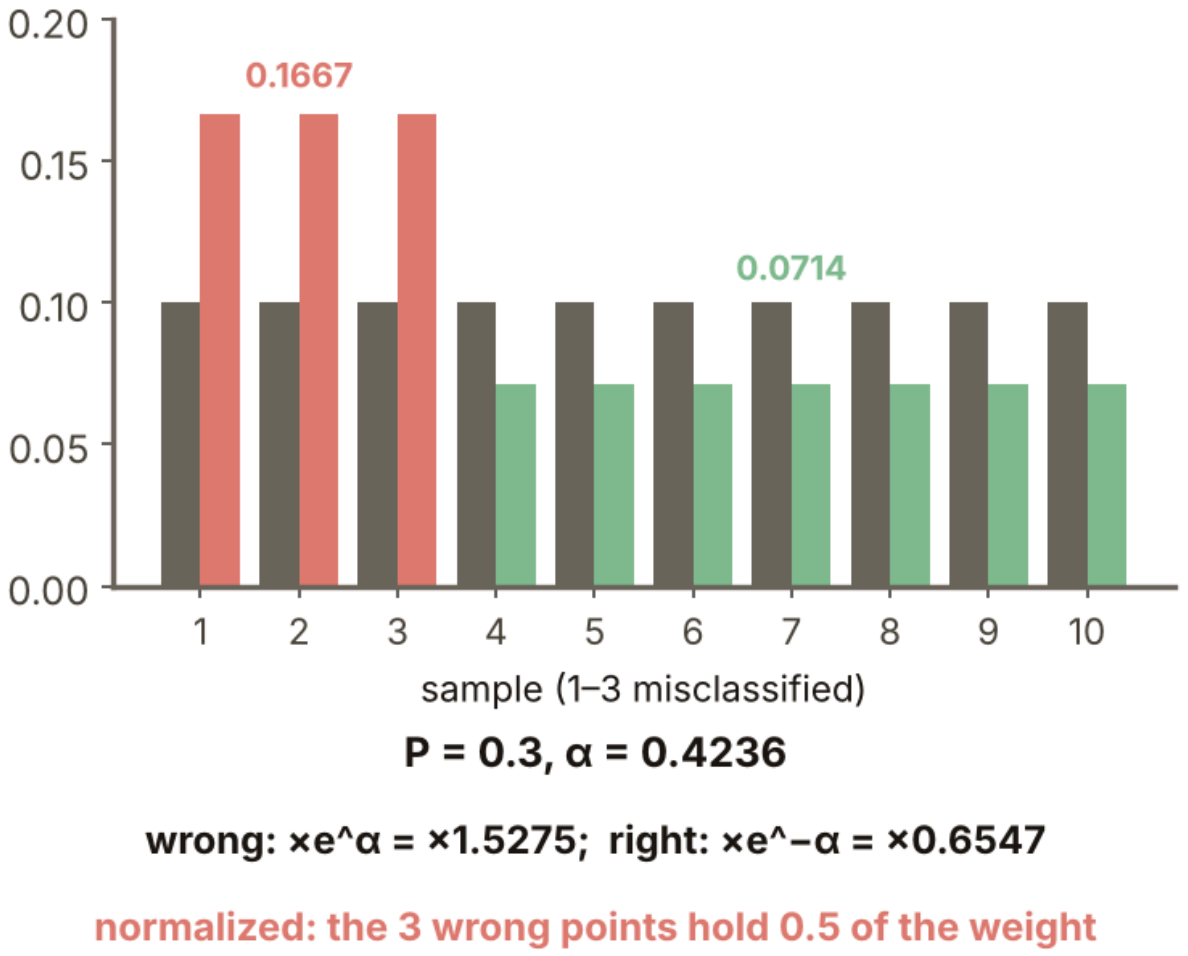
P → 0: a large voice; P = 0.5: no voice (α = 0)

P > 0.5: negative α, use the answer inverted

4. The weight update

$w_i \leftarrow w_i \exp(-y_i \alpha_m \phi_m(x_i))/Z_m$; wrong points ($y\phi < 0$) grow and right ones shrink. **By hand** (designed here): 10 points of weight 0.1, 3 wrong: $P = 0.3$, $\alpha = 0.4236$; wrong ones times $e^{\alpha} = 1.5275$ (0.1528), right ones times $e^{-\alpha} = 0.6547$ (0.0655); $Z = 0.9165$. Normalized: wrong ones 0.1667 each, right ones 0.0714 each; the three wrong points together hold exactly 0.5 and the next ϕ must look after them first.

Figure `lec05-kp25-update`: the 10 weights before and after.



5. The algorithm

$w_i = 1/N$; repeat: find ϕ_m minimizing P_m, compute α_m, update and normalize the weights; finally $f(x) = \operatorname{sign} \sum_k \alpha_k \phi_k(x)$. **Setup** (Example 4.3 redone here): two 20-D Gaussians, 200 training and 400 test points; each round picks the best stump over all features and thresholds; 2000 rounds. Round 1: training error 25.5% and test error 28.75%, better than an arbitrary fixed stump (mean 31.6%).

Figure `lec05-kp25-algo`: the AdaBoost algorithm.

start: $w_i = 1/N$

repeat:

find φ_m minimizing P_m (weighted error)

$\alpha_m = \frac{1}{2} \ln((1 - P_m) / P_m)$

$w_i \leftarrow w_i \exp(-y_i \alpha_m \varphi_m(x_i))$, then divide by Z_m

until a stopping rule is met

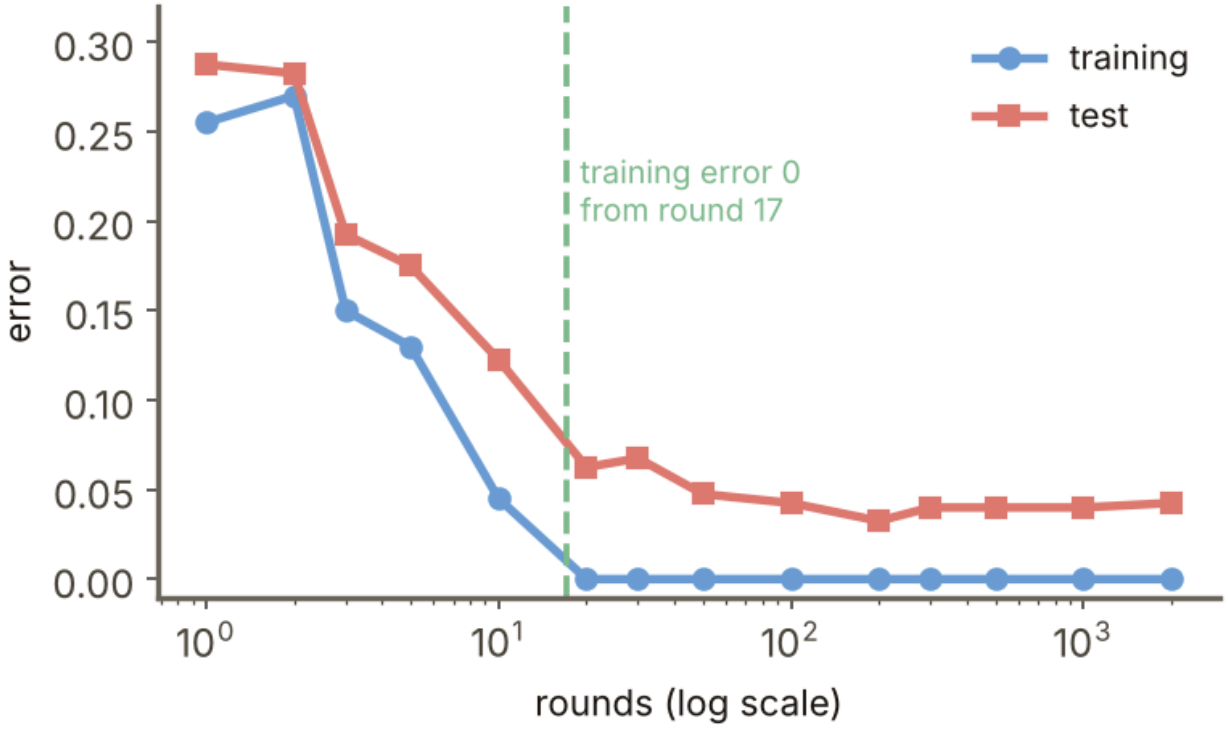
$f(x) = \text{sign}(\Sigma_k \alpha_k \varphi_k(x))$

Example 4.3: 20-D Gaussians, 200 training, 400 test points, base classifier = a one-question stump

6. The error curves

Training error: 25.5% at round 1, 4.5% at round 10 and 0 from round 17 on. Test error: 28.75% (1), 12.25% (10), 4.75% (50), 3.25% (200), 4.0% (500), 4.25% (2000). After the training error reaches zero the test error keeps falling, then stays near 4% after about 200 rounds, flat within sampling error (with 400 test points 1% is only 4 points).

Figure `lec05-kp25-curve`: training and test error against rounds (log axis).



test error: 28.7% (1), 12.2% (10), 3.25% (200), 4.25% (2000)

after training error reaches 0 the test error keeps falling

to about round 200, then stays flat near 4%

7. The margin explanation

Normalized margin $\text{margin} = yF(x)/\sum_k \alpha_k \in [-1, 1]$: positive only if correct, larger means more confidence. The bound (Schapire et al.): generalization error $\lesssim \text{prob}\{\text{margin} < \gamma\} + O(\sqrt{V_c/(N\gamma^2)})$, independent of the number of rounds K; this is the source of the immunity. **Experiment** (designed here), smallest margin: -1.000 at round 1, -0.205 at 10, $+0.148$ at 50, $+0.203$ at 200, $+0.220$ at 2000; share with margin < 0.1: 25.5%, 8.5%, 0%. The criticism: the bound is very loose (unless N is in the tens of thousands) and only a qualitative explanation; stage-wise optimization is itself an implicit regularization.

Figure `lec05-kp25-margin`: the smallest margin and test error at different rounds.

rounds	min margin	margin < 0.1	test
1	-1.000	25.5%	28.75%
10	-0.205	8.5%	12.25%
50	+0.148	0.0%	4.75%
100	+0.179	0.0%	4.25%
200	+0.203	0.0%	3.25%
2000	+0.220	0.0%	4.25%

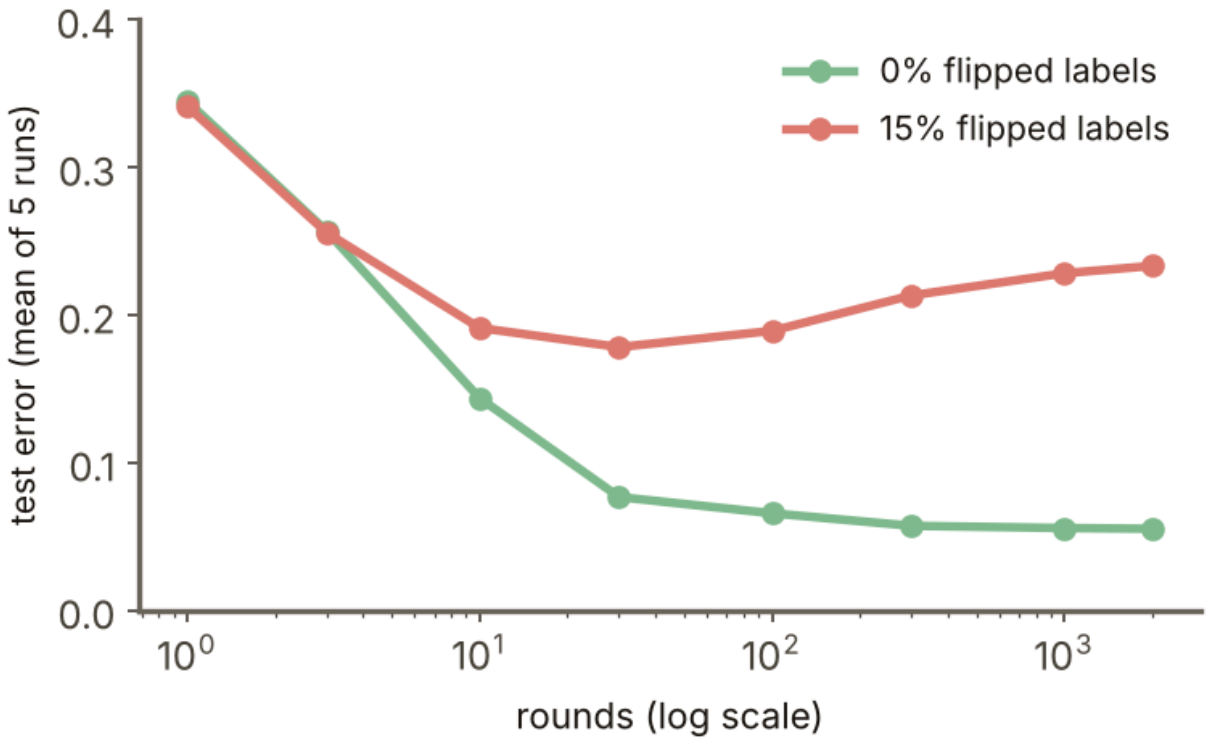
margin = yF(x) / Σα, in [−1, 1], positive when correct

boosting keeps raising the smallest margins

8. No panacea

Example (designed here): data of the same kind with 15% of the training labels flipped at random; means of 5 fresh draws for 0% and 15% (so unlike the single run before; test labels are clean). Test error: clean: 14.4% (10 rounds) → 6.6% (100) → 5.5% (2000); 15% flipped: 19.1% → 18.9% → 23.3%, lowest 17.8% (30 rounds). A likely cause: the exponential loss punishes bad samples with large negative margins too much; after 2000 rounds the 15.5% flipped samples hold 36% of the weight (one run). **Common misconception**: AdaBoost never overfits; on hard tasks with a high Bayes error and noisy labels it can degrade sharply (source). **To remember**: weak classifiers + weighting the points that are wrong + a weighted vote; the weights fall out of the exponential loss and α is set by the weighted error.

Figure `lec05-kp25-noise`: test error with clean labels and with 15% flipped.



clean labels: 5.5% at the end

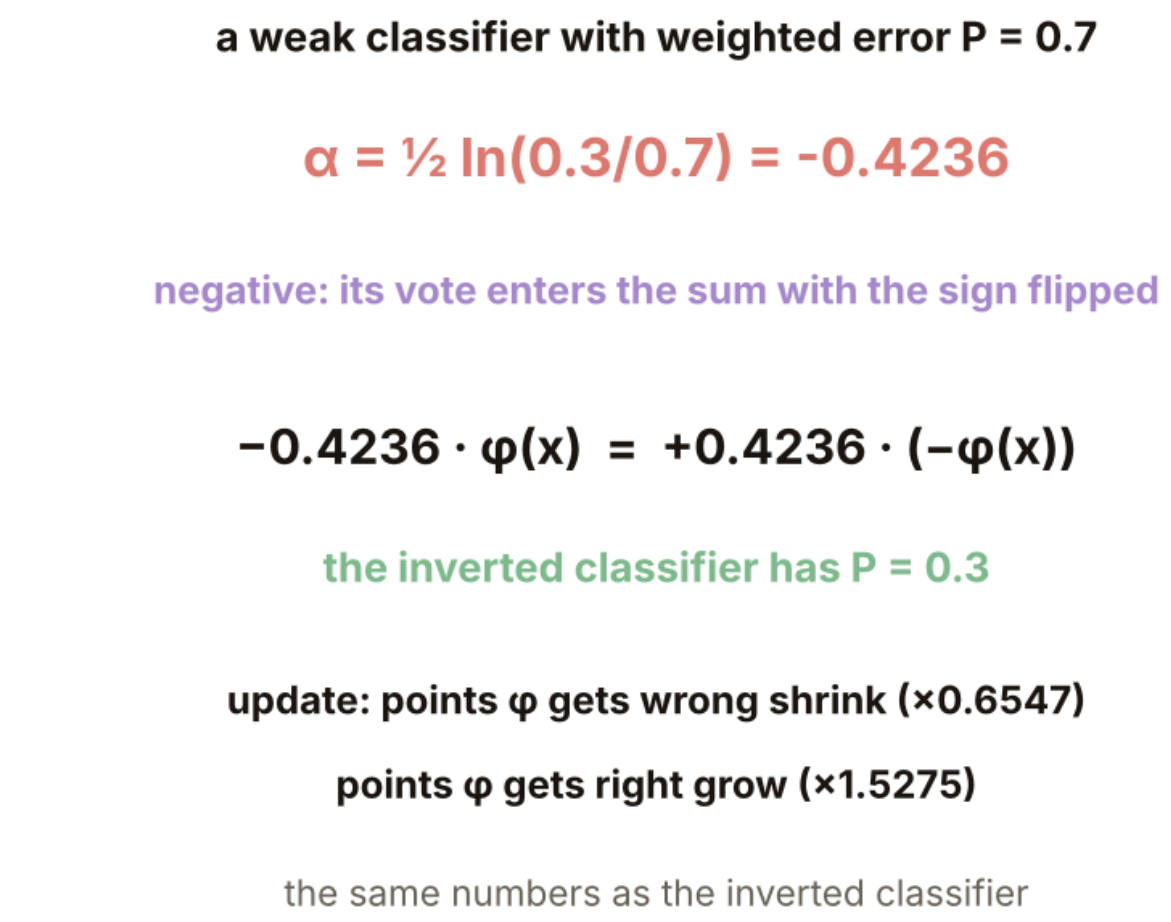
15% flipped: best 17.8% at round 30, 23.3% at 2000

after 2000 rounds 36% of the weight sits on the 15.5% flipped points

Extension: negative α, practice and worked solution

A negative α: for $P = 0.7$, $\alpha = \frac{1}{2}\ln(0.3/0.7) = -0.4236$; $-0.4236\,\phi = +0.4236\,(-\phi)$ and the weighted error of $-\phi$ is 0.3. Points ϕ gets wrong are multiplied by $e^{-0.4236} = 0.6547$ (shrink) and points it gets right by 1.5275 (grow), exactly as for the inverted classifier.

Figure `lec05-kp25-neg`: a negative α and the inverted classifier.



Practice: 5 samples with weights 0.2 each, 1 misclassified by ϕ, weighted error $P = 0.2$.

1. What is α?
2. After the update and normalization, what is the weight of each sample?

Figure `lec05-kp25-practice`: the setup of the practice.

5 samples with equal weights 0.2; one is misclassified

weighted error P = 0.2

question 1: α for P = 0.2

question 2: new weights after the update and normalization

$$w_i \leftarrow w_i \exp(-y_i\, \alpha\, \varphi(x_i)) \,/\, Z$$

Worked solution of questions 1 and 2: (1) $\alpha = \frac{1}{2} \ln \frac{0.8}{0.2} = \frac{1}{2} \ln 4 = 0.6931$; $e^{\alpha} = 2$ and $e^{-\alpha} = 0.5$. (2) The wrong one $0.2 \times 2 = 0.4$ and each of the 4 right ones $0.2 \times 0.5 = 0.1$; $Z = 0.8$; normalized: 0.5 for the wrong one and 0.125 for each right one. (3) Check: the total is $0.5 + 4 \times 0.125 = 1$; the wrong point holds exactly half, the same pattern as the 3 points holding 0.5 on page 4: after the update the old ϕ has weighted error exactly 0.5, so a new ϕ is needed. If the stump family is closed under negation the search simply picks $-\phi$ ($P = 0.3$), so a negative α is rare.

Figure `lec05-kp25-practice-sol`: the computation.

$$\alpha = \tfrac{1}{2}\ln(0.8/0.2) = \tfrac{1}{2}\ln 4 = 0.6931$$

$$e^{\alpha} = 2, \quad e^{-\alpha} = 0.5$$

$$\text{wrong: } 0.2 \times 2 = 0.4; \ \text{right } (\times 4)\text{: } 0.2 \times 0.5 = 0.1 \text{ each}$$

$$Z = 0.4 + 4 \times 0.1 = 0.8$$

$$\text{normalized: wrong } 0.5, \ \text{right } 0.125 \text{ each}$$

$$\text{the misclassified sample now holds half the weight}$$

Terms and source

中文	English
弱分類器	Weak classifier
決策樁	Stump
逐級最佳化	Stage-wise optimization
正規化邊距	Normalized margin

Source: Chapter 4, KP25, §4.22 (read 2026-10-25). The boosting question for weak learners, the exponential loss and stage-wise optimization, the origin of the weights, the α formula, the weight update, the algorithm, the margin explanation and the criticism that the bound is loose, the fact that boosting is no panacea and the 'to remember' line follow the source; the stump errors, the 10-point hand run, the error curves and margin table of the redone Example 4.3, the 15% label-flip experiment, the negative α and the practice were designed and recomputed for this lecture ([added], (program-computed)).